

UNIVERSITÉ DE LA RÉUNION
UFR SCIENCES ET TECHNOLOGIES
RAPPORT DE STAGE DE MASTER M2 INFORMATIQUE
LABORATOIRE D'INFORMATIQUE ET DE MATHÉMATIQUES

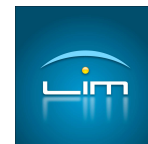
Développement d'un moteur de recommandation

Auteur :
Andrianambinina Marius
RABENARIVO
n° étudiant : 36007553

Encadrant :
Dr. Matthieu BAGORY

Responsable de stage :
Dr. Dominique GAY

Période du stage : 09 janvier 2017 - 30 juin 2017



Remerciements

En préambule de ce rapport, je tiens tout d'abord à vivement remercier le personnel administratif la Scolarité, de l'UFR Sciences et Technologies et du Laboratoire d'Informatique et de Mathématiques de l'Université de La Réunion grace auxquels mes études dans l'établissement se sont bien déroulé en bon et du forme.

Ensuite, je tiens à témoigner de ma profonde gratitude au corps enseignant de la formation de Master 2 que j'ai suivi. Ils se sont montré professionnels et compréhensifs tout en prodiguant des cours aussi inspirants les uns que les autres.

Je ne saurais ne pas remercier le Directeur du Laboratoire d'Informatique et de Mathématique en la personne du Pr. Jean DIATTA qui a bien voulu me faire confiance pour ce projet de stage qui arrive bientôt à son terme. Je tiens également à vivement remercier le Pr. Frédéric MESNARD qui a tenu le rôle de responsable pédagogique de la formation Master 2. Qui dans son rôle nous a accompagné avec rigueur dans toutes les démarches par lesquelles nous sommes passés.

Je tiens de même à exprimer mes sincères remerciements à mon encadreur Dr. Matthieu BAGORY et à mon responsable de stage Dr. Dominique GAY, MCF qui se sont montré à l'écoute et compréhensifs en m'accompagnant tout au long du stage.

Je voudrais aussi prendre un moment pour remercier la famille du Pr. Jean TAINÉ et Jean-Luc MARTAIL ainsi que Nirisoa RAHERINAINA qui par leur soutien financier et administratif m'ont permis de poursuivre mes études à l'Université de La Réunion.

Je tiens également à remercier ma famille, mes proches et amis qui m'ont témoigné de leur amitié et soutien tout au long de mes études, pendant mon stage, ainsi que lors de la rédaction de ce rapport.

À maman,

Résumé

Pleazup est un réseau social pour partager ses listes d'envies d'idées cadeaux. Le stage a pour but de développer un moteur de recommandation ayant pour objectif de proposer aux utilisateurs des idées cadeaux originales d'après leur profil et leurs interactions sociales. La première partie du stage a consisté à développer une recommandation statique en calculant périodiquement des métriques associés à chaque idée cadeau, à partir des données sociales disponibles. La deuxième partie consiste à mettre en place une recommandation dynamique en faisant coopérer les utilisateurs. La troisième partie du stage consiste à faire de la recommandation basée sur les caractéristiques des idées cadeaux : données textes et images.

Mots-clés

: stage, master 2, recherche & développement, moteur de recommandation

Abstract

Pleazup is a social network to share his wish list of gift ideas. The object of the internship is to develop a recommendation engine whose aim is to offer users original gift ideas based on their profile and social interactions. The first part of the internship consisted of developing a static recommendation by periodically calculating metrics associated with each gift idea according to the available social data. The second part is to put in place a dynamic recommendation by the use of user-based collaborative filtering technique. The third part of the internship consists of making the recommendation based on the characteristics of gift ideas : text data and images.

Keywords

: internship, master 2, research & development, recommendation engine

Table des figures

1.1	Organigramme du LIM	3
1.2	Stack technologique de Pleazup	4
1.3	Parcourir la liste des envies de cadeau des proches	4
1.4	Suggérer des idées cadeaux anonymement pour être sûr(e) qu'elles plaisent	5
1.5	Être notifié(e) en avance de l'anniversaire des proches	5
1.6	Réserver anonymement une idée pour être seul(e) à l'offrir	6
1.7	Avoir des recommandations d'idées cadeaux basées sur ses interactions antérieures	6
2.1	Principales classifications des systèmes de recommandation	9
2.2	Filtrage collaboratif	11
3.1	Architecture système de PredictionIO avec Universal Recommender	16
3.2	Vue d'ensemble de la plate-forme PredictionIO	17
3.3	Vue d'ensemble de l'architecture d'une instance de moteur	18
3.4	Comparaison de LLR avec d'autres mesures de similarité par rapport à leur précision (MAP@K)	20
3.5	Perspective de haut niveau du Lambda Architecture	20
3.6	Interface web de l'application Pleazup	21
3.7	Comparaison de prix entre différents fournisseurs d'hébergement Cloud relativement à la quantité de mémoire vive fournie	22
3.8	Architecture système de l'écosystème technologique avec le système de recommandation	31

Liste des tableaux

2.1	Les quatre fonctions essentielles de la recommandation automatique pour les utilisateurs	8
3.1	Input Events	26
3.2	IndexedDataset de Mahout	26
3.3	Sortie de SimilarityAnalysis.cooccurrences pour P^tV	27
3.4	Sortie de SimilarityAnalysis.cooccurrences pour P^tP	27
3.5	Matrice de contingence	27

Table des matières

0.1	Introduction	1
1	Contexte général du projet et Entreprise d'accueil	2
1.1	Entreprise d'accueil	2
1.1.1	Pleazup	2
1.1.2	Le Laboratoire d'Informatique et de Mathématiques (LIM)	2
1.2	Sujet du stage	3
1.3	Organisation et planification	4
1.4	Fonctionnalités de <i>Pleazup</i>	4
2	Les systèmes de recommandation	7
2.1	Principales fonctionnalités des systèmes de recommandation	7
2.2	Exigences pour les systèmes de recommandation [1]	8
2.3	Classification des systèmes de recommandation [2]	8
2.4	Méthodes retenues pour l'implémentation d'un moteur de recommandation :	
	Méthodes évolutives de voisinage basées sur la similarité	9
2.4.1	MapReduce	9
2.4.2	Exposition du problème	10
2.4.3	Modèle de calcul	10
2.4.4	Approche séquentielle	11
2.4.5	Algorithme proposé	12
3	Mise en œuvre Technologique	15
3.1	PredictionIO	15
3.1.1	Historique de PredictionIO	15
3.1.2	Architecture système de PredictionIO	15
3.1.3	Vue d'ensemble de PredictionIO	16
3.1.4	Achitecture DASE d'un moteur de recommandation dans PredictionIO	17
3.1.5	L'Universal Recommender	18
3.1.6	L'Algorithme Correlated Cross-Occurrence (CCO)	19
3.1.7	Historique et Motivations derrière la genèse de LLR	19
3.1.8	Lambda Architecture	20
3.2	Implémentation du moteur de recommandation	20
3.2.1	Recommandation statique	20
3.2.2	Choix du fournisseur de service Cloud	21
3.2.3	Filtrage collaboratif	21
3.2.4	Règles métier	28
3.2.5	Filtrage basé sur le contenu	30
3.2.6	Implémentation et mise en production	30
3.3	Evaluation et test de montée en charge	30
3.3.1	Evaluation	30
3.3.2	Test de montée en charge	31

Conclusion	32
Bibliographie	33
Annexes	36
Annexe A : Accomplissements personnels de l'étudiant	36

0.1 Introduction

D’après l’ethnologue Marcel Mauss, nous sommes soumis à une triple obligation : celle de donner, mais aussi de recevoir et de rendre cadeau pour cadeau. [3] L’écrivain-psychologue française Maryse Vaillant, affirme de même que “C’est pourquoi un cadeau raté nous blesse tant. Celui-ci révèle à quel point le donateur nous connaît mal, et dévoile l’image dans laquelle il essaie de nous enfermer” en parlant du fait que le cadeau cherche à symboliser le bénéficiaire ainsi que le donateur. [4]

Face au choix de cadeau qui peut donc s’avérer crucial, le réseau social *Pleazup* propose à l’utilisateur de leur apporter une assistance dans leur choix de cadeau quand arrive le moment d’offrir.

C’est dans ce cadre que j’ai choisi de mener mon stage de fin d’étude pour le développement d’un moteur de recommandation intégré au réseau social pour partager ses listes d’envies d’idées cadeaux, avec la collaboration du Laboratoire d’Informatique et de Mathématiques de l’Université de La Réunion et l’accompagnement de l’Incubateur Régional de La Réunion.

Mon choix s’est porté sur ce sujet pour son attrait théorique en voulant répondre à la question : Comment recommander des idées cadeaux à une personne en se basant sur ses interactions précédentes sur le réseau social ? Cela m’a permis d’appliquer l’état de l’art de la recherche sur l’apprentissage automatique (*Machine Learning*) et les systèmes de recommandation sur une problématique réelle pour laquelle un réseau social s’est donné pour mission d’apporter une solution.

Pour bien situer le sujet dans son cadre, nous allons commencer par la présentation de *Pleazup* et de ses fonctionnalités ainsi que le Laboratoire LIM et l’organisation qui a été mis en place pour suivre et gérer l’avancement du stage. A la suite de cela, seront présentés les systèmes de recommandation avec les fonctionnalités attendues ainsi que les défis que de tels systèmes doivent adresser, pour ensuite exposer la méthode retenue pour l’implémentation du moteur de recommandation. Dans la dernière partie du rapport, nous allons voir les technologies mises en œuvre ainsi que l’implémentation et la mise en production du système de recommandation qui a été développé au cours du stage avant de conclure.

Chapitre 1

Contexte général du projet et Entreprise d'accueil

1.1 Entreprise d'accueil

1.1.1 Pleazup

Pleazup est un réseau social de partage et d'échange d'idées de cadeau entre proches qui a vu le jour en 2014. D'abord commencé à Paris par Matthieu Bagory et Diane Frachon, l'application a été incubée à la Technopole de La Réunion depuis le 15 décembre 2015.

La Technopole de La Réunion est une association ayant pour objet de favoriser les échanges entre les entreprises innovantes, les organismes de formation et les laboratoires de recherche sur le territoire de La Réunion. [5]

L'Incubateur Régional de La Réunion accompagne les porteurs de projets innovants de manière personnalisée. L'Incubateur Régional est agréé par le Ministère de l'Enseignement Supérieur et de La Recherche, et soutenu financièrement par l'Europe, l'Etat, la Région Réunion et la CINOR. [6]

Pour son incubation Pleazup a été adossé par le laboratoire d'Informatique et de Mathématiques de l'Université de La Réunion.

1.1.2 Le Laboratoire d'Informatique et de Mathématiques (LIM)

Le LIM est un laboratoire de recherche de l'Université de La Réunion, situé au Parc Technologique Universitaire (PTU) de Saint-Denis. Il est structuré en trois axes :

- Epistémologie et Didactique de l'Informatique et des Mathématiques (EDIM)
- Informatique et Applications (ITA)
- Mathématiques (MATHS)

Le LIM développe, à travers ses membres, des partenariats académiques aux niveaux local, régional (zone Océan Indien), national et international, ainsi que des partenariats industriels locaux. Il participe à la formation par la recherche, adosse le Master Informatique et Mathématiques de l'UFR Sciences et Technologies, le département Informatique et Télécommunications de l'ESIROI (Ecole Supérieure d'Ingénieurs Réunion-Océan Indien), et il co-adosse le master MEEF (Métiers de l'Enseignement, de l'Éducation et de la Formation) de l'ESPE (École Supérieure du Professorat et de l'Éducation). Il est membre de l'Ecole doctorale STS (Sciences Technologies Santé) et de la fédération de recherche OMNCG (Observatoire des milieux naturels et des changements globaux), de l'UR. [7]

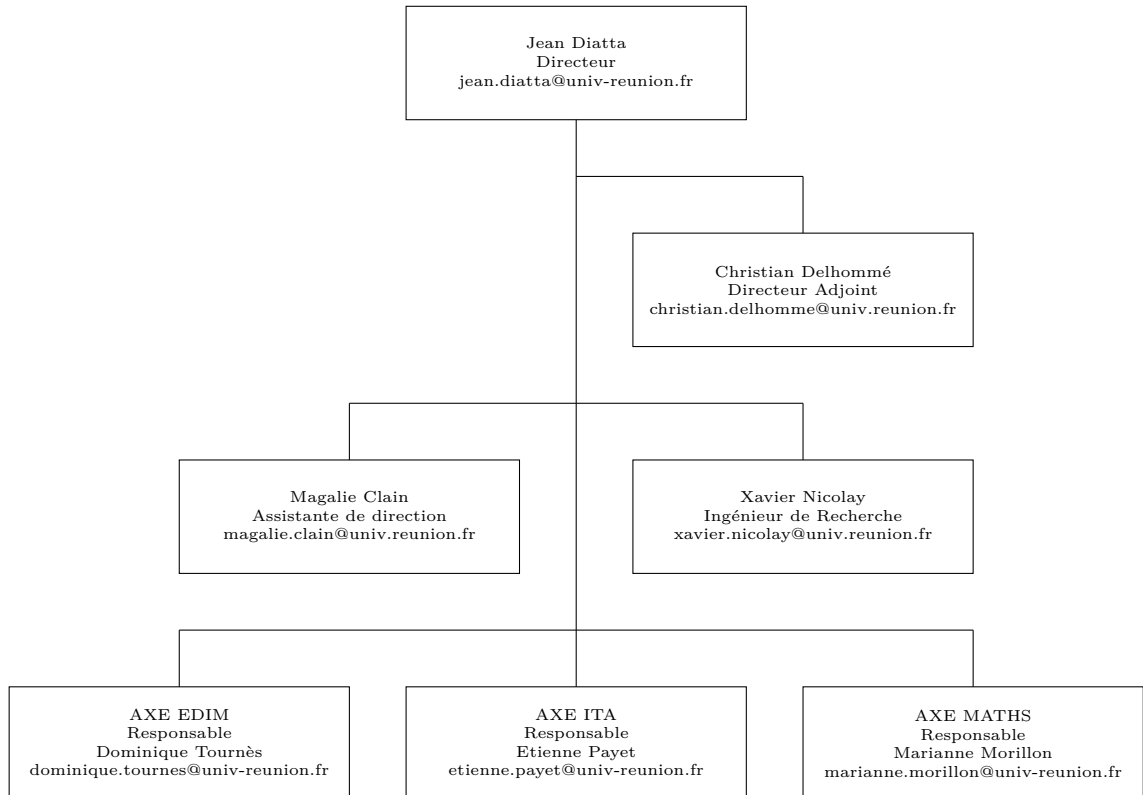


FIGURE 1.1: Organigramme du LIM

1.2 Sujet du stage

Le stage a pour objectif la conception et l'implémentation d'un moteur de recommandation. Il s'agira dans un premier temps de développer une recommandation générale/statique basée sur les caractéristiques des idées cadeaux et exploitant les données sociales associées (like, nombre de suggestion, nombre d'acceptation/refus, nombre de réservation, nombre d'achats, etc).

Dans un deuxième temps, l'objectif sera de développer une recommandation individuelle/dynamique basée sur les caractéristiques des utilisateurs et exploitant la ressemblance de leur centre d'intérêt estimé à partir de leurs idées (nom, description, etc) et de leurs données sociales (acceptation/refus, réseau d'amis, etc).

Si le temps le permet, un dernier objectif sera d'utiliser une API de classification automatique des images (clarifai.com, cloud.google.com/vision/, etc) afin d'améliorer l'estimation des centres d'intérêt des utilisateurs.

L'implémentation devra être compatible avec le stack technologique en JavaScript (hébergement Heroku, environnement Node.js, plateforme de backend Parse) qui est représenté sur la figure 1.2. [8]

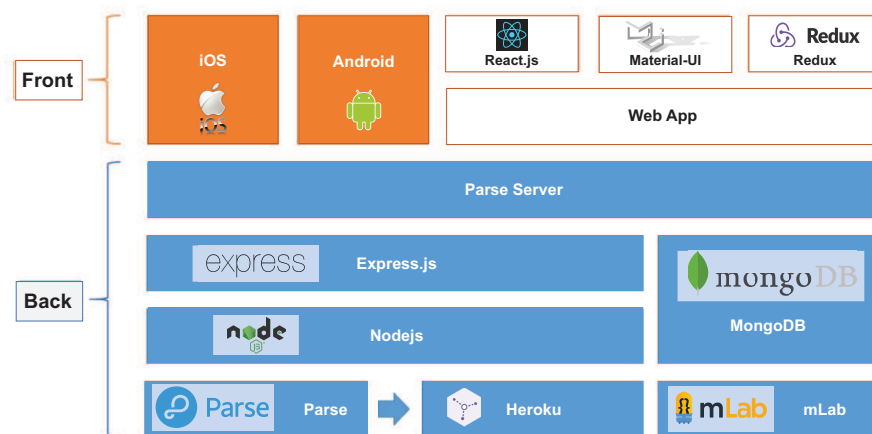


FIGURE 1.2: Stack technologique de Pleazup

1.3 Organisation et planification

Nous avons mis en place une réunion hebdomadaire avec le responsable du stage et l'enseignant référent. Dans cette réunion, les progrès de la semaine précédente sont exposés et les problèmes rencontrés discutés pour ensuite définir les priorités de la semaine en cours.

La planification a déjà été établie dans le sujet de stage et son accomplissement est géré par la procédure expliquée ci-dessus. Jusqu'ici les problèmes rencontrés ont pu être résolus à temps ou on a trouvé des approches pour les résoudre qui sont planifiés pour les futures tâches à faire.

1.4 Fonctionnalités de *Pleazup*

L'application *Pleazup* permet de

FIGURE 1.3: Parcourir la liste des envies de cadeau des proches

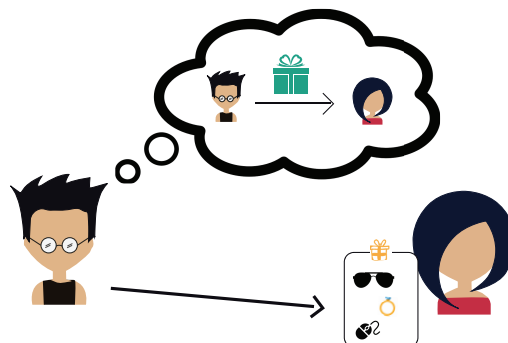


FIGURE 1.4: Suggérer des idées cadeaux anonymement pour être sûr(e) qu'elles plaisent

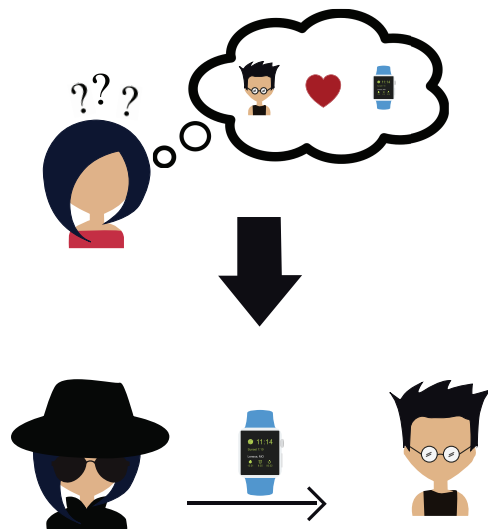


FIGURE 1.5: Être notifié(e) en avance de l'anniversaire des proches

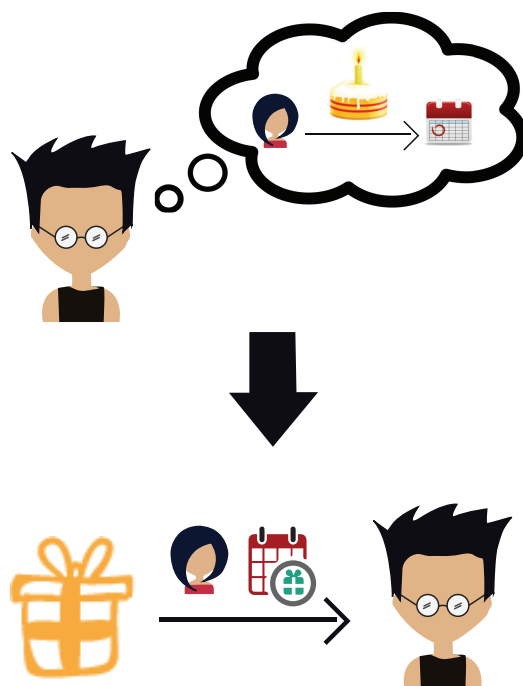


FIGURE 1.6: Réserver anonymement une idée pour être seul(e) à l'offrir

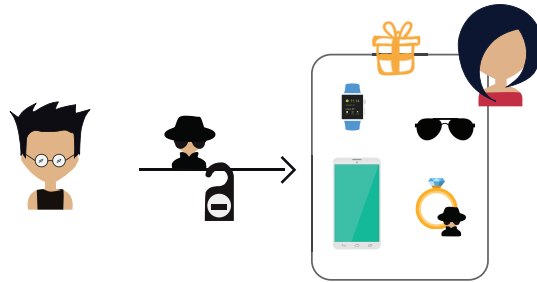
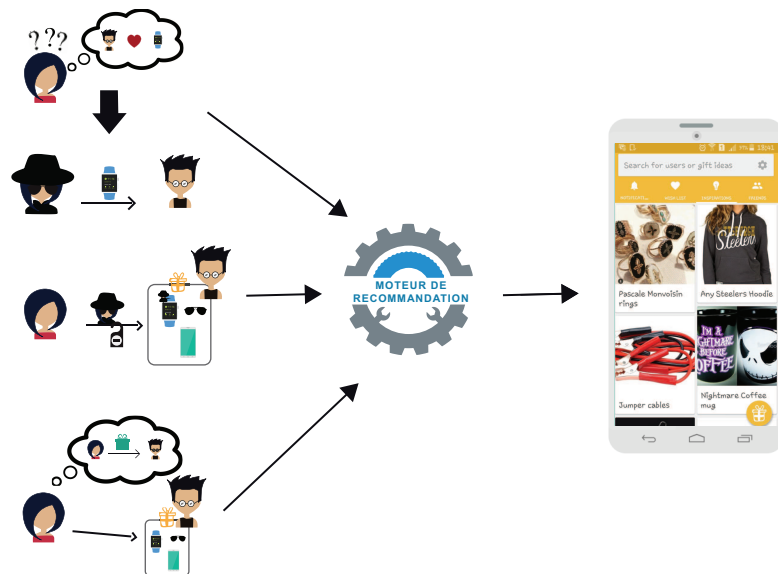


FIGURE 1.7: Avoir des recommandations d'idées cadeaux basées sur ses interactions antérieures



Chapitre 2

Les systèmes de recommandation

Dans la vie quotidienne, les gens s'appuient sur les recommandations d'autres personnes par le bouche à oreille, des lettres de référence, des nouvelles des médias, des enquêtes générales, des guides de voyage, etc. Les systèmes de recommandation assurent et augmentent ce processus social naturel pour aider les gens à filter des livres, des articles, des pages Web, des films, de la musique, des restaurants, des blagues, des produits d'épicerie, etc., afin de trouver les informations les plus intéressantes et les plus précieuses pour eux. [9]

Dans ce chapitre, nous allons voir d'abord les fonctionnalités attendues des systèmes de recommandation. Ensuite, les exigences pour les systèmes de recommandation seront énumérées avant de donner une classification des systèmes de recommandation.

2.1 Principales fonctionnalités des systèmes de recommandation

Bien que la littérature académique se concentre sur la prédiction du score¹, la priorisation par l'industrie des fonctionnalités est très différente : les recommandations générées elles-mêmes et leur utilité sont plus importantes.

Nous proposons 4 fonctions essentielles qui sont :

- la prédiction de score,
- la prédiction de rang, arrangement d'articles par ordre de pertinence,
- la recommandation contextuelle, non personnalisée pour un utilisateur anonyme,
- et la recommandation personnalisée d'articles.

Le tableau 2.1 décrit ces fonctions de manière plus formelle.

1. Étant donné un article, le système donne une estimation de la note que pourrait donner un utilisateur à celui-ci

Fonction essentielle	Description	Fonction cœur de la recommandation
Aide à la Décision	Soit 1 article i , un utilisateur u veut savoir s'il va apprécier i	Prediction de score
Aide à la Comparaison	Soient n articles, un utilisateur u veut savoir quel article choisir	Prédiction de rang
Aide à l'Exploration	Soit 1 article i , un utilisateur u veut savoir quels sont les k articles similaires	recommandation article à articles
Aide à la Découverte	Compte tenu d'un énorme catalogue d'articles, l'utilisateur u veut trouver k nouveaux articles intéressants	Recommandation personnalisée

TABLE 2.1: Les quatre fonctions essentielles de la recommandation automatique pour les utilisateurs

2.2 Exigences pour les systèmes de recommandation [1]

- L'adaptabilité à de nombreuses sources de données car les systèmes industriels doivent pouvoir être déployés sur différents services avec différents types de données,
- La robustesse à des données bruitées ou éventuellement corrompues, car certaines sources de données peuvent être plus ou moins fiables,
- Gestion du démarrage à froid au moment du lancement du système de recommandation : le démarrage à froid est le problème pour le système de commencer la prédiction sur les utilisateurs ou les articles sans données suffisantes (les prédictions peuvent être médiocres, voire impossibles),
- La capacité à passer à l'échelle car les systèmes opérationnels doivent pouvoir faire face à des données volumineuses,
- La réactivité du système, car le temps de réponse et les problèmes d'apprentissage en ligne sont également cruciaux,
- Relation de confiance, basée sur la transparence et l'explication : la transparence d'un système de recommandation est un facteur important pour l'acceptation des recommandations,
- La gestion du "long tail", c'est-à-dire la gestion de tous les articles qui ne sont pas souvent achetés / vus mais que nous voulons dans certains cas promouvoir.

2.3 Classification des systèmes de recommandation [2]

Plusieurs typologies de système de recommandation sont connues dans la littérature.

- la typologie classique, avec le filtrage collaboratif, le filtrage basé sur le contenu et le filtrage hybride
- la typologie de Su et Khoshgoftarr, 2009 [9], restreint au filtrage collaboratif
- la typologie de Rao et Talwar, 2008 [10], qui se base sur les sources de données utilisées par les systèmes.

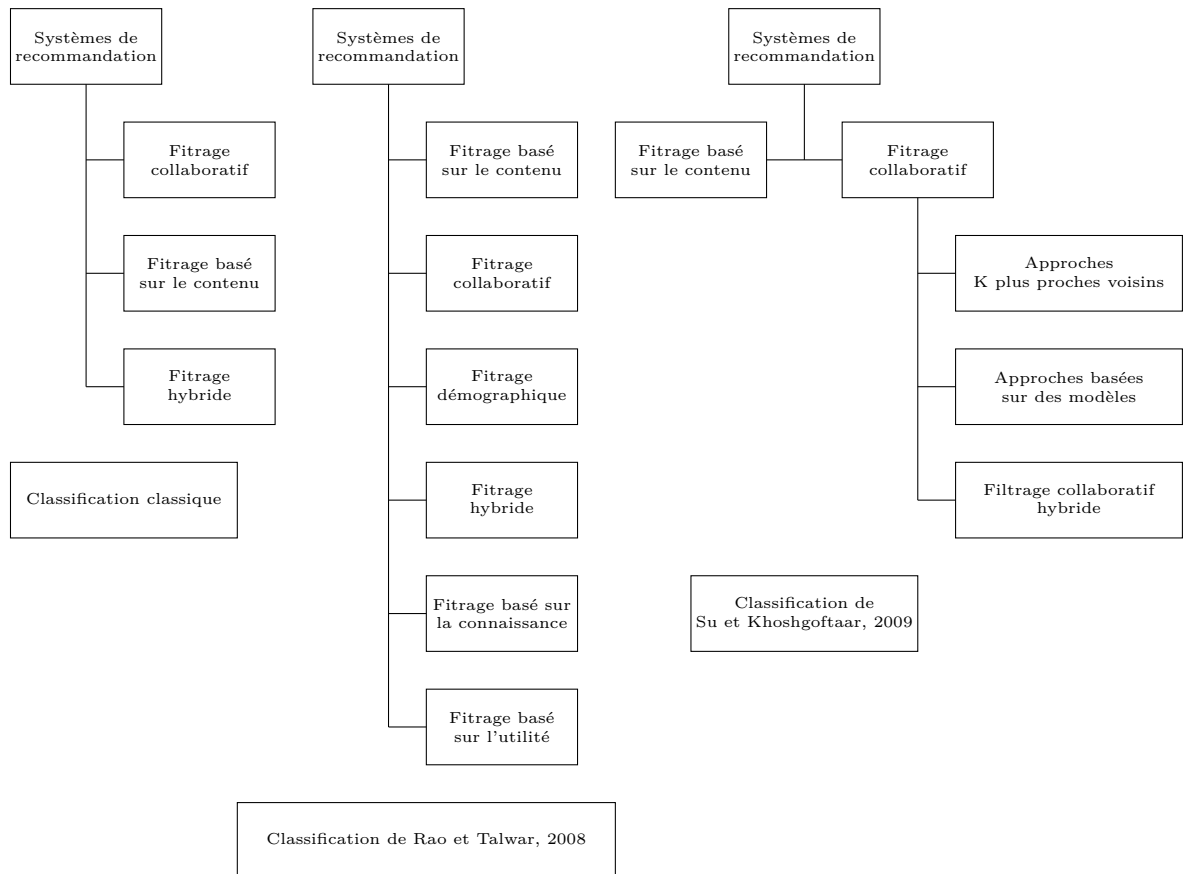


FIGURE 2.1: Principales classifications des systèmes de recommandation

2.4 Méthodes retenues pour l'implémentation d'un moteur de recommandation : Méthodes évolutives de voisinage basées sur la similarité

Avec une croissance de la taille de données de plus en plus rapide, l'efficacité du traitement, l'évolutivité des systèmes et les calculs sous-jacents constituent une préoccupation majeure.

Dans un environnement de production, les calculs hors ligne nécessaires à l'exécution d'un système de recommandation doivent être exécutés périodiquement dans le cadre de flux de travail analytiques plus importants et sont donc sous contrainte de temps et de ressources.

Pour des raisons économiques et opérationnelles, il n'est souvent pas souhaitable d'exécuter ces calculs hors connexion sur une seule machine : cette machine pourrait tomber en panne et, avec des tailles de données croissantes, des mises à jour matérielles constantes pourraient être nécessaires pour améliorer les performances de la machine pour respecter les contraintes de temps. En raison de ces inconvénients, une solution à une seule machine peut rapidement devenir coûteuse et difficile à utiliser.

2.4.1 MapReduce

Afin de résoudre ce problème, les progrès récents dans le traitement des données à grande échelle proposent d'exécuter des calculs analytiques à forte intensité de données de manière parallèle et tolérante aux pannes sur un grand nombre de machines standards.

Cela permettra de rendre l'exécution indépendante des défaillances individuelles de la

machine et permettra en outre d'augmenter la capacité de calcul en ajoutant simplement plus de machines au cluster, évitant ainsi la nécessité d'une mise à jour constante du matériel sur une machine unique.

Lorsqu'il est appliqué aux systèmes de recommandation, cette approche technique nécessite la reformulation des algorithmes existants pour leur permettre d'utiliser une plate-forme de traitement en parallèle.

2.4.2 Exposition du problème

Soit A une matrice $|U| \times |I|$ contenant toutes les interactions connues entre un ensemble d'utilisateurs U et un ensemble d'articles I .

Un utilisateur u est représenté par son historique d'interactions avec les articles $a_{u\bullet}$, la u -ième ligne de A .

Le top N des recommandations pour cet utilisateur correspond aux premiers N éléments sélectionnés à partir d'un classement r_u de tous les articles selon la façon dont ils seraient préférés par l'utilisateur. Ce classement est déduit des schémas trouvés dans A .

2.4.3 Modèle de calcul

Notations

$a_{u\bullet}$ désigne la u -ième rangée de la matrice d'interaction A , $a_{\bullet i}$ désigne la i -ième colonne de A , $|U|$ désigne le nombre d'utilisateurs qui est égal au nombre de lignes dans A . *foreach* $i \in v$ désigne l'itération sur les indices des entrées non nulles d'un vecteur v , *foreach* $(i, k) \in v$ désigne l'itération sur les indices et les valeurs non nulles correspondantes d'un vecteur v .

Afin d'avoir une image plus claire de l'approche par voisinage, il est utile d'exprimer l'algorithme en termes d'opérations d'algèbre linéaire. Les méthodes basées sur le voisinage trouvent et classent les articles qui ont été préférés par d'autres utilisateurs qui partagent des parties de l'historique des interactions $a_{u\bullet}$. Soit A une matrice binaire avec $A_{ui} = 1$ si un utilisateur u a interagi avec l'article i et $A_{ui} = 0$ sinon. Pour une comparaison par paire entre les utilisateurs, un produit scalaire des lignes de A donne le nombre d'éléments que les utilisateurs correspondants ont en commun. De même, un produit scalaire des colonnes de A donne le nombre d'utilisateurs qui ont interagi avec les deux articles correspondant aux colonnes.

Lors du calcul des recommandations pour un utilisateur particulier avec un filtrage collaboratif basé sur l'utilisateur, d'abord une recherche d'autres utilisateurs ayant un goût similaire est menée. Cela se traduit par la multiplication de la matrice A par l'historique d'interaction de l'utilisateur $a_{u\bullet}$, ce qui aboutit à un classement de tous les utilisateurs. Deuxièmement, la préférence de l'utilisateur actif pour un article est estimée en calculant la somme pondérée des préférences de tous les autres utilisateurs pour cet élément et le classement correspondant. Dans notre simple modèle, cela se traduit par la multiplication du classement de tous les utilisateurs par A^T . Cela signifie que l'approche complète peut être résumée par les deux multiplications suivantes :

$$r_u = A^T(Aa_{u\bullet}) \quad (2.1)$$

Pour exploiter la plus grande stabilité des relations entre les articles, une autre variante des méthodes de voisinage appelée filtrage collaboratif basé sur les articles a été développée, qui examine d'abord les articles et évalue leurs cooccurrences. Cette approche calcule une matrice de similarités d'élément à élément et permet des recommandations rapides car le modèle ne doit pas être recalculé pour de nouveaux utilisateurs. L'expression de l'approche

basée sur les articles se traduit par le fait de simplement déplacer les parenthèses dans notre formule, car $A^T A$ donne exactement la matrice des cooccurrences des éléments :

$$r_u = (A^T A)a_{u\bullet} \quad (2.2)$$

Cela montre que le filtrage collaboratif basé sur les utilisateurs et celui basé sur les articles partagent le même modèle de calcul fondamental.

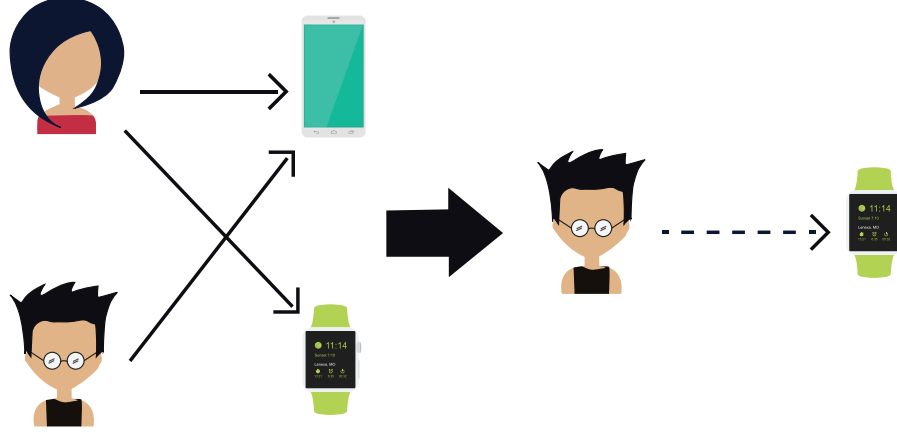


FIGURE 2.2: Filtrage collaboratif

2.4.4 Approche séquentielle

L'approche séquentielle standard pour calculer la matrice de similarité des articles $S = A^T A$ est illustrée dans l'algorithme 1

Algorithm 1 Approche séquentielle pour calculer la matrice de cooccurrences des articles

```

for all item  $i$  do
  for all user  $u$  who interacted with  $i$  do
    for all item  $j$  that  $u$  also interacted with do
       $S_{ij} = S_{ij} + 1$ 
    end for
  end for
end for

```

Pour chaque article i , nous devons rechercher chaque utilisateur qui ont interagi avec i . Ensuite, nous itérons sur les autres éléments j de l'historique des interactions de u et enregistrons la cooccurrence de i et j .

Nous pouvons exprimer mathématiquement l'approche en utilisant 3 sommations imbriquées :

$$S = A^T A = \sum_{i=1}^{|I|} \sum_{u=1}^{|U|} \sum_{j=1}^{|I|} A_{i,u}^T \cdot A_{u,j} \quad (2.3)$$

Si nous souhaitons distribuer le calcul sur plusieurs machines d'un cluster distribué, cette approche ne peut être réalisée de manière efficace car elle nécessite un accès aléatoire aux utilisateurs et aux articles dans ses boucles internes. Son modèle d'accès aléatoire ne peut être réalisé efficacement lorsque nous devons travailler sur des données partitionnées.

De plus, à première vue, la complexité de l’approche basée sur les articles est quadratique en le nombre d’articles, car chaque article doit être comparé à chaque autre article. Cependant, la matrice d’interaction A est habituellement très creuse. Il est fréquent que seule une petite fraction de toutes les cellules soit connue et que le nombre d’éléments non nuls dans A soit linéaire en le nombre de lignes de A . Ce fait limite sévèrement le nombre de paires d’articles à comparer, car seules les paires qui partagent au moins un utilisateur qui a interagi avec les deux doivent être prises en considération. Cela diminue la complexité de l’algorithme en quadratique en nombre d’éléments non nuls dans la ligne la plus dense plutôt que quadratique en nombre de colonnes de la matrice A . Le coût de l’algorithme peut alors être exprimé comme la somme du traitement du carré du nombre d’interactions de chaque utilisateur unique.

Malheureusement, les jeux de données de filtrage collaboratif partagent une propriété commune aux jeux de données générés par les interactions humaines : le nombre d’interactions par utilisateur suit une distribution à queue lourde, ce qui signifie que le traitement d’un petit nombre de “power users” domine le coût de l’algorithme.

Schelter et al. ont élaboré une formulation parallélisable du calcul pour étendre cette approche sur une plate-forme de traitement parallèle. [11] Comme nous devons utiliser un système de fichiers distribué, l’algorithme proposé doit pouvoir fonctionner avec des données d’entrée partitionnées. En outre, il doit se mettre à l’échelle linéairement par rapport à un nombre croissant d’utilisateurs. Ils ont permis l’utilisation d’un large éventail de mesures de similarité et ont ajouté des moyens pour gérer les coûts de calcul introduits par les “power users”.

2.4.5 Algorithme proposé

Nous allons maintenant présenter le framework algorithmique proposé. Nous commençons par montrer comment effectuer le comptage distribué des cooccurrences des articles pour un modèle simple qui utilise des données binaires. Ensuite, nous généralisons l’approche à des données non binaires et qui permet l’utilisation d’une grande variété de mesures de similarité. Enfin, nous discutons des moyens de sparsification de la matrice de similarité, et des moyens pour faire des recommandations sur des lots et d’appliquer un down sampling sélectif pour obtenir une évolutivité linéaire avec un nombre croissant d’utilisateurs.

Afin de mettre à l’échelle le calcul de similarité présenté à l’Algorithme 1, il doit être exprimé comme un algorithme parallèle, pour rendre sa vitesse d’exécution proportionnelle au nombre de machines dans le cluster.

La solution consiste à réorganiser les boucles de l’Algorithme 1 pour obtenir la formulation du produit vectoriel de la matrice de la multiplication matricielle. Parce que la u -ième colonne de A^T est identique à la u -ième ligne de A , nous pouvons calculer S en ayant seulement besoin d’accéder aux lignes de A :

$$S = A^T A = \sum_{i=1}^{|I|} \sum_{u=1}^{|U|} \sum_{j=1}^{|I|} A_{i,u}^T \cdot A_{u,j} = \sum_{u=1}^{|U|} a_{u\bullet} (a_{u\bullet})^T \quad (2.4)$$

À la suite de cette constatation, nous partitionnons A par ses lignes (les utilisateurs) et la stockons dans le système de fichiers distribué. Chaque fonction **map** lit une seule ligne de A , calcule le produit vectoriel de la ligne avec lui-même et envoie la matrice intermédiaire résultante ligne par ligne sur le réseau. La fonction **reduce** doit simplement sommer tous les résultats partiels, calculant ainsi une ligne de S par invocation (Algorithme 2).

Cette approche nous permet d’exploiter la sparsité des matrices intermédiaires du produit vectoriel en faisant en sorte que la fonction **map** ne renvoie que les entrées non nulles. En même temps, nous appliquons une fonction **combine** (identique à la fonction

Algorithm 2 Calcul des cooccurences des articles

```
function MAP( $a_{u\bullet}$ )
  foreach  $i \in a_{u\bullet}$  do
     $c \leftarrow \text{SPARSE\_VECTOR}()$ 
    foreach  $j \in a_{u\bullet}$  with  $j > i$  do
       $c[j] \leftarrow 1$ 
    end for
    EMIT( $i, c$ )
  end for
end function
function COMBINE( $i, c_1, \dots, c_n$ )
   $c \leftarrow \text{VECTOR\_ADD}(c_1, \dots, c_n)$ 
  EMIT( $i, c$ )
end function
function REDUCE( $i, c_1, \dots, c_n$ )
   $s \leftarrow c_1, \dots, c_n$ 
  EMIT( $i, s$ )
end function
```

reduce) sur les vecteurs émis par les fonctions `map`, ce qui fait que le système minimise la quantité de données qui doit être envoyée sur le réseau. En outre, nous ne calculons que la moitié triangulaire supérieure de S , car la matrice de similarité résultante est symétrique.

Calcul de similitude généralisé

Les jeux de données du monde réel contiennent des représentations plus riches pour encoder les interactions des utilisateurs qu'un simple codage binaire. Ils sont soit des retours explicites, comme des notes numériques que les utilisateurs choisissent sur une échelle prédéfinie ou des retours implicites que l'on obtient en comptant le nombre de fois qu'un comportement particulier, comme un clic sur un lien ou une vue de page, ait été observé. Nous devons pouvoir choisir parmi une variété de mesures de similarité pour comparer ces interactions entre les articles, afin de pouvoir trouver celle qui reflète le mieux les relations inhérentes aux données.

Schelter et al. ont incorporé dans l'algorithme un large éventail de mesures pour comparer les interactions de deux articles i et j en utilisant trois fonctions canoniques. [11]

Comme nous allons le voir par la suite, une mesure appelée Log-Likelihood Ratio s'est montré plus efficace que les autres dans la pratique et a été retenue pour l'implémentation de l'algorithme que l'on peut trouver dans le framework d'apprentissage automatique que nous avons utilisé pour implémenter le moteur de recommandation.

Sparsification

Afin de pouvoir traiter des cas avec un nombre énorme d'articles, nous ajoutons des moyens pour diminuer la densité de la matrice de similarité S à notre mise en œuvre finale.

Pour éliminer les paires avec une similarité proche de zéro, on peut spécifier un seuil de similarité, pour lequel on évalue une contrainte de taille en vue d'éliminer les paires d'éléments de score plus faible au début du processus [12] et éventuellement supprimer toutes les entrées de S qui sont inférieures au seuil.

En outre, il a été démontré que la qualité de prédiction de l'approche basée sur les articles est suffisante si seule la fraction supérieure des éléments similaires est utilisée. [13] Par conséquent, nous ajoutons une autre étape de *MapReduce* qui conserve uniquement

ces éléments similaires par article, en une unique passe sur les données.

Down sampling sélectif pour une évolutivité linéaire

Rappelons que notre objectif est de développer un framework algorithmique qui évolue linéairement par rapport à une base d'utilisateurs croissante. Le coût de l'approche basée sur les articles est dominé par les lignes les plus denses de A , qui correspondent aux utilisateurs ayant le plus d'interactions. Ce coût, que nous exprimons en tant que nombre de cooccurrences d'éléments à considérer, est la somme des carrés du nombre d'interactions de chaque utilisateur.

Si nous examinons seulement le fait qu'un utilisateur ait interagi ou pas avec un article, alors intuitivement, nous ne tirerions pas beaucoup d'informations utiles d'un "power user" : chaque élément supplémentaire avec lequel il interagit coïncidera avec la grande quantité d'objets qu'il préférerait. Nous nous attendons à obtenir plus d'informations auprès des utilisateurs ayant moins d'interactions, mais avec un goût très varié.

Suite à ce raisonnement, Schelter et al. proposent d'appliquer ce que qu'on appelle une *interaction-cup* : Consistant à sous échantillonner de manière sélective l'historique des interactions des "power users". [11]

Nous appliquons cela en échantillonnant au hasard p interactions de l'historique de chaque utilisateur, limitant ainsi le nombre maximum d'interactions par utilisateur dans l'ensemble des données à p .

Notez que cet échantillonnage est uniquement appliqué au petit groupe de "power users". Cela n'affecte pas les données fournies par la grande majorité des utilisateurs non-"power users" dans la "long tail". Limiter l'effort par utilisateur de cette façon limite le coût global de notre approche à $|U|p^2$.

Schelter et al. ont montré expérimentalement qu'un p de taille modérée est suffisant pour obtenir une qualité de prédiction proche de celle obtenue des données non échantillonnées. [11]

Chapitre 3

Mise en œuvre Technologique

3.1 PredictionIO

3.1.1 Historique de PredictionIO

À l'origine appelé TappingStone [14] quand il a été fondé en 2012 à Londres par son co-fondateur et PDG Simon Chan et les autres co-fondateurs Donald Szeto, Kenneth Chan et Thomas Stone, PredictionIO a commencé comme start-up qui a d'abord construit un produit qu'il décrit comme "Machine Learning as a Service" avant de pivoter vers un modèle open source.

PredictionIO a ensuite rapidement pris de l'importance et a même été classé sur Github en tant que projet de Machine Learning basé sur Apache Spark le plus populaire dans le monde.

Depuis que ça ait été annoncé le 29 septembre 2015, ActionML a été le partenaire officiel de PredictionIO qui fournit un soutien à la communauté et aux clients. Formé par les scientifiques précédents et le gestionnaire de produits de PredictionIO, l'équipe ActionML a une connaissance approfondie et une expérience avec la plate-forme PredictionIO et continue d'être engagée dans la base de code.

Il a ensuite été acquis par Salesforce le 19 février 2016 [15] avant d'être donné à l'Apache Software Foundation (ASF) le 25 juillet de la même année [16] et à l'unanimité, la plate-forme a été acceptée dans le programme ASF incubator. Cela démontre la reconnaissance par la communauté open source de l'importance du projet PredictionIO.

Le 17 mai 2017, le Project Management Committee (PMC) pour Apache PredictionIO (incubating) a demandé à Sara Asher de devenir gestionnaire de produit, et elle a accepté.

Sara est directrice de Product Management pour Salesforce Einstein, où elle crée des produits qui permettent aux gens de créer des applications plus intelligentes avec Salesforce et Advanced AI. Avant Salesforce, Sara a travaillé chez Alpine Data où elle était chef de produit et directrice fondatrice de Alpine Labs. Sara détient un AB en mathématiques de l'Université de Princeton et un doctorat en mathématiques de l'Université Northwestern.

Être un gestionnaire de produits permet la gestion des tickets JIRA. Cela devrait rendre plus prioritaire les fonctionnalités du produit.

Au moment de la rédaction de ce rapport, la communauté de PredictionIO avec Sara Asher comme gestionnaire de produit prépare le graduation du projet en tant que projet Apache de premier plan.

3.1.2 Architecture système de PredictionIO

Prediction IO est un :

- Serveur de Machine Learning open source ;

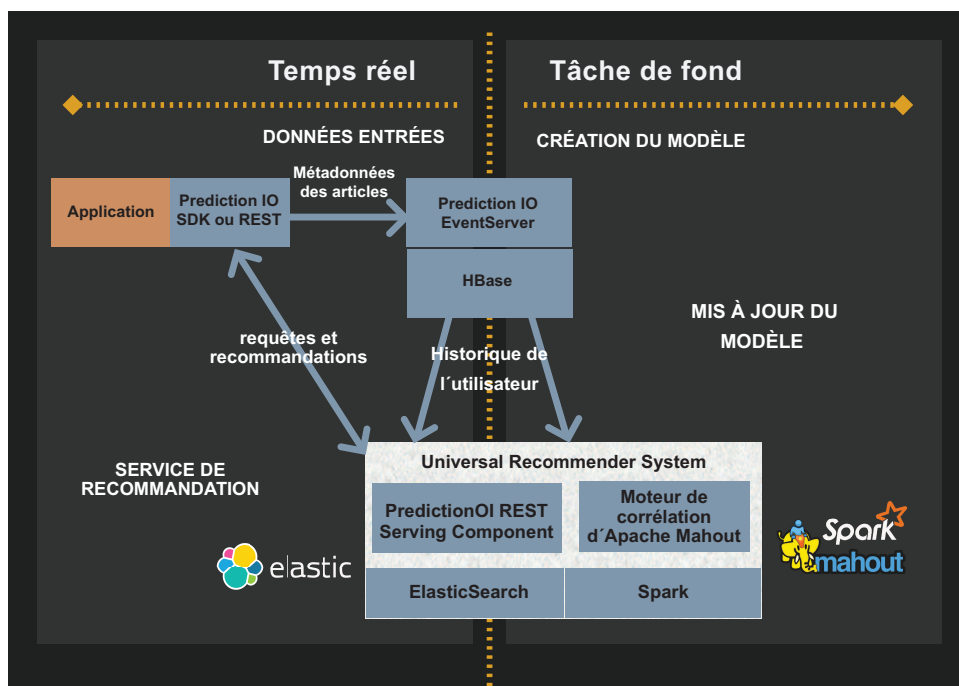


FIGURE 3.1: Architecture système de PredictionIO avec Universal Recommender

- Construit au-dessus d'un stack open source intégrant les technologies de l'état de l'art :
 - Apache Spark,
 - Apache Mahout,
 - Apache HBase,
 - Spray et
 - Elasticsearch ;
- et qui implémente un Lambda Architecture (voir ci-dessous 3.1.8).

Apache HBase L'Event Server utilise Apache HBase qui est une base de données open-source, distribuée, versionnée et non relationnelle [17] en tant que magasin de données. Il stocke les événements importés.

Apache Spark Spark est un moteur de traitement de données à grande échelle qui gère la préparation des données et l'entrée de l'algorithme, l'entraînement et parfois le traitement du serving.

Elasticsearch stocke les métadonnées telles que les versions des modèles, les versions du moteur, les clés d'accès et les mapping d'identification de l'application, les résultats de l'évaluation, etc. Pour certains modèles de moteur comme l'Univeral Recommender, il stocke également le modèle issu de l'entraînement.

Spray est une boîte à outils open-source pour la construction de couches d'intégration REST / HTTP en Scala et Akka. Spray est asynchrone, basé sur le principe d'acteur, rapide, léger, modulaire et vérifiable.

Apache Mahout est un projet de la Fondation Apache visant à fournir un environnement pour créer rapidement des applications d'apprentissage automatique évolutives.

3.1.3 Vue d'ensemble de PredictionIO

PredictionIO se compose des composants suivants :

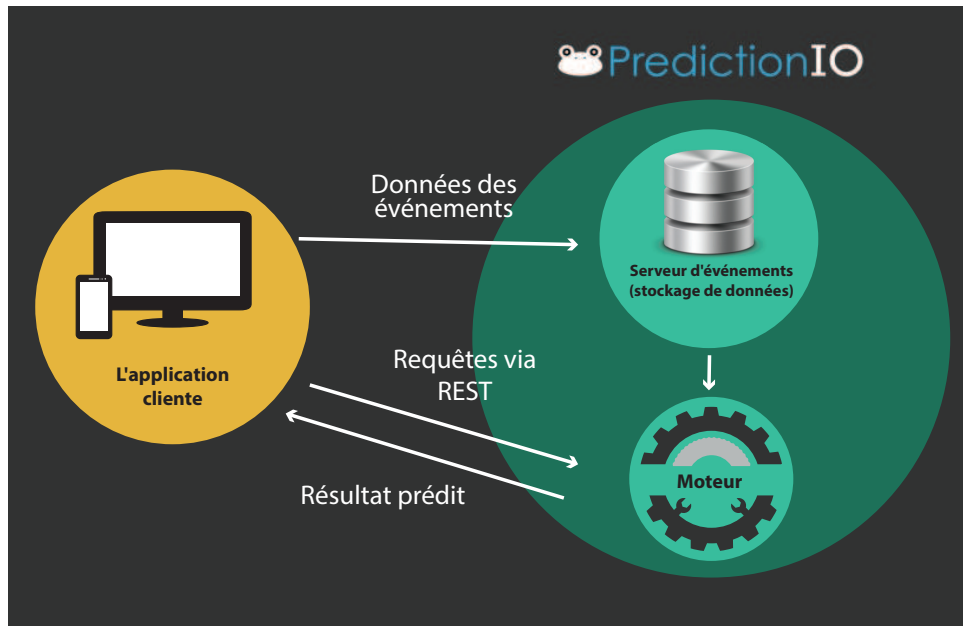


FIGURE 3.2: Vue d'ensemble de la plate-forme PredictionIO

- Plate-forme PredictionIO - stack open source pour le développement, l'évaluation et le déploiement de moteurs avec des algorithmes d'apprentissage automatique.
- Serveur d'événements - couche d'analyse d'apprentissage automatique open source pour unifier des événements provenant de plates-formes multiples
- Galerie de templates - l'endroit où on peut télécharger des modèles de moteurs pour différents types d'applications d'apprentissage automatique

Serveur d'événements

Dans un scénario commun, le serveur d'événements de PredictionIO recueille continuellement les données de l'application. Un moteur PredictionIO construit alors des modèles prédictifs avec un ou plusieurs algorithmes utilisant les données. Une fois qu'il est déployé en tant que service Web, il écoute les requêtes de l'application et répond avec les résultats prédits en temps réel.

Le serveur d'événements collecte les données de l'application, en temps réel ou en batch. Il peut également unifier les données liées à l'application à partir de plusieurs plates-formes.

Une fois les données recueillies, elles servent principalement à deux fins :

- Fournir des données au(x) moteur(s) pour l'entraînement et l'évaluation du modèle
- Offre une vue unifiée pour l'analyse des données

3.1.4 Architecture DASE d'un moteur de recommandation dans PredictionIO

Le moteur est responsable de faire des prédictions. Il contient un ou plusieurs algorithmes d'apprentissage automatique. Il lit les données d'entraînement et construit des modèles prédictifs. Il est ensuite déployé en tant que service Web. Un moteur déployé répond en temps réel aux requêtes de prédiction de l'application via REST API.

La galerie de modèles de moteur de PredictionIO propose des modèles de moteurs pour toutes sortes de tâches d'apprentissage automatique. On peut facilement créer un ou plusieurs moteurs à partir de ces modèles.

Les composants d'un modèle, à savoir Data Source, Data Preparator, Algorithm(s), Serving et Evaluation, sont tous personnalisables pour des besoins spécifiques. [18]

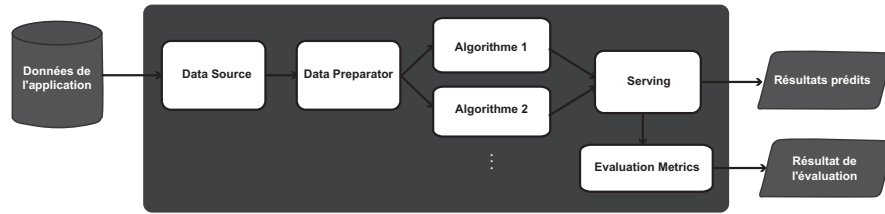


FIGURE 3.3: Vue d'ensemble de l'architecture d'une instance de moteur

Le code d'un moteur se compose de composants D-A-S-E : [19]

[D] Data Source et Data Preparator

Data Source lit les données à partir d'une source d'entrée et les transforme en un format souhaité. Data Preparator prétraite les données et les envoie à l'algorithme pour l'entraînement du modèle

[A] Algorithm

Le composant Algorithm comprend l'algorithme de Machine Learning et ses paramètres. Il détermine comment un modèle prédictif est construit.

[S] Serving

Le composant Serving prend des requêtes de prédiction et renvoie les résultats de prédiction. Si le moteur comporte plusieurs algorithmes, Serving combinera les résultats en un seul. En outre, une logique métier spécifique peut être ajoutée dans la section Serving pour personnaliser davantage les résultats finalement obtenus.

[E] Evaluation Metrics

Les Evaluation Metrics quantifient la précision de la prédiction avec un score numérique. Ils peuvent être utilisés pour comparer des algorithmes ou des paramètres d'un algorithme.

3.1.5 L'Universal Recommender

L'Universal Recommender (UR) est un nouveau type de système de recommandation collaboratif basé sur un algorithme qui peut utiliser des données provenant d'une large variété d'indicateurs de goût de l'utilisateur qui s'appelle l'algorithme Correlated Cross-Occurrence. Contrairement à la factorisation de matrice que l'on peut trouver dans les autres algorithmes comme l'ALS de MLlib, l'algorithme CCO de l'UR est capable d'ingérer n'importe quel nombre d'actions des utilisateurs, événements, données de profil, et informations sur le contexte. Il sert ensuite les résultats d'une manière rapide et se prête facilement à la mise à l'échelle. Il supporte aussi les propriétés des articles pour le filtrage et le boosting des recommandations et peut alors être considéré comme un système de recommandation hybride collaboratif et basé sur le contenu.

L'utilisation des données de multiple types change fondamentalement la façon dont un système de recommandation est utilisé et, lorsqu'il est employé correctement, fournira une augmentation significative de la qualité des recommandations vs. en utilisant un seul événement utilisateur. La plupart des systèmes de recommandation, par exemple, peuvent

seulement utiliser les événements "achat". En utilisant tout ce que nous connaissons sur un utilisateur et son contexte nous pouvons mieux prédire ses préférences. [20]

3.1.6 L'Algorithme Correlated Cross-Occurrence (CCO)

Pour la majeure partie de l'histoire des systèmes de recommandation, la science des données ne pouvait trouver que des moyens d'utiliser que un parmi les types d'indicateurs de préférence utilisateur disponibles. Plus précisément, il s'agissait d'un type par application, mais il y a tellement plus que nous savons du comportement de l'utilisateur notamment sous forme de "clickstream"¹ qui restaient inutilisés.

L'algorithme Correlated Cross-Occurrence (CCO) a été avancé pour découvrir quel comportement d'un utilisateur donné est en corrélation avec le type d'action que l'on souhaite recommander. Si vous souhaitez recommander d'acheter, de jouer, de regarder ou de lire, est-il possible que d'autres choses connues d'un utilisateur correspondent à cette action recommandée ? Des choses comme une vue de page, une préférence pour une catégorie, l'emplacement enregistré, l'appareil utilisé, les vues des détails d'un article, ou tout autre chose connue sur l'utilisateur. En outre, comment évaluer la corrélation ?

C'est là que le Log-Likelihood Ratio (LLR) entre en jeu -un test probabiliste pour la corrélation entre 2 événements. Ceci est très important car il n'y a pas de relation linéaire entre les types d'événements. La corrélation est au niveau de l'utilisateur et des événements individuels et c'est là que LLR excelle. Pour illustrer cela, mettons nous dans une situation de E-commerce avec 1/2 des vues menant à un achat ? Vous pourriez le penser, mais si l'utilisateur a vu 2 choses et a acheté l'une d'elles, la corrélation est de 100 % pour l'une des vues et 0 % pour l'autre. Ainsi, certaines données de vue sont utiles pour prédire les achats et d'autres sont inutiles. LLR est un test très respecté pour ce type de corrélation. [21]

3.1.7 Historique et Motivations derrière la genèse de LLR

Pat Ferrel dans le cadre d'une consultance dans une entreprise de RaaS a amélioré la précision du système de recommandation de 12% mais des problèmes étaient restés [22] :

- ca n'utilisait que les données "achat"/conversion et on jetait 100x des données de vue et autres comportements de l'utilisateur
- ce n'était pas "temps réel"
- ALS ne marche pas bien que la cooccurrence
- beaucoup trop de "edge cases", des utilisateurs qui n'avaient pas de recommandations
- ne s'adapte pas aux metadonnées/contenus des articles

Pat raconte que dans des discussions avec Ted Dunning (un mentor d'Apache Mahout, où on ont été obtenues les recommandations de cooccurrence et d'ALS), ce dernier aurait décrit une façon de comparer 2 événements au niveau individuel. La comparaison est appelée Log-Likelihood Ratio ou LLR. Cela nous permettrait de regarder toutes les pages vues et de voir ceux qui correspondent aux achats. Toutes les vues de page ne sont donc pas créées égales, mais nous avons maintenant un moyen de trouver les plus importantes. [21]

La figure 3.4 a été générée en utilisant les anciens recommandeurs de Mahout Mapreduce, qui avaient des paramètres de similarité enfichables. Pat Ferrel l'a fait tourner sur un grand ensemble de données E-Commerce à partir d'un vrai site d'e-commerce. Les données étaient pour 6 mois de ventes. Ils ont procédé à la validation croisée d'un ensemble d'entraînement de 80 et à un ensemble de test / essai de 20%. L'ensemble de tests était constitué de 20% des ventes les plus récentes. Ils ont alors mesuré le MAP@k pour plusieurs

1. Une série de clics faite pendant la navigation sur un site web

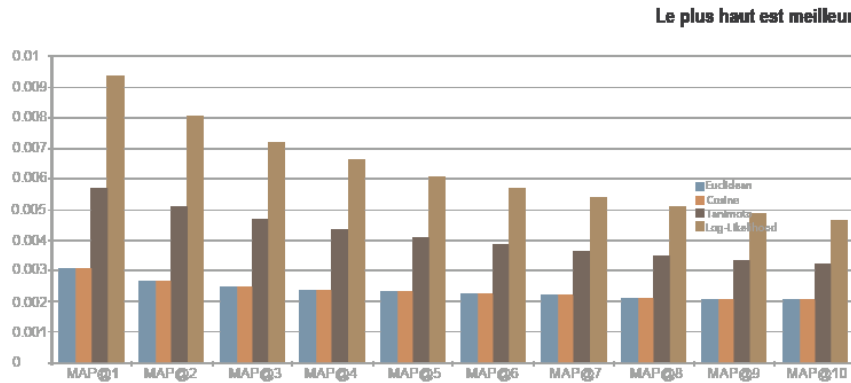


FIGURE 3.4: Comparaison de LLR avec d'autres mesures de similarité par rapport à leur précision (MAP@K)

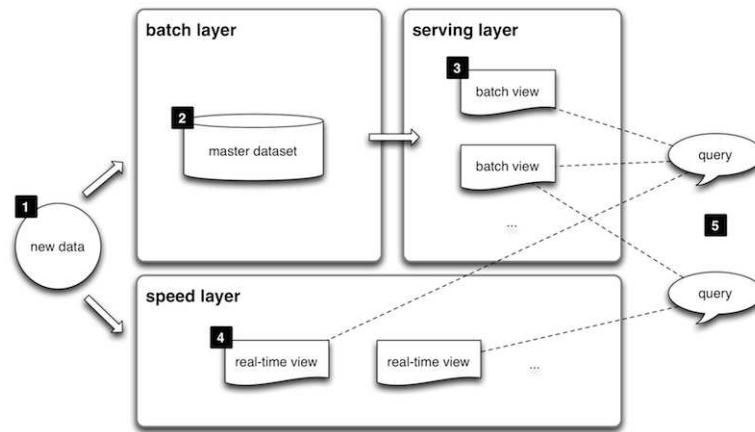


FIGURE 3.5: Perspective de haut niveau du Lambda Architecture

k. Une baisse dans MAP@k quand k augmente signifie que le classement des éléments est correct. Plus MAP@k est grand meilleure est la précision des recommandations. [23]

3.1.8 Lambda Architecture

Nathan Marz a introduit le terme Lambda Architecture (LA) pour une architecture de traitement de données générique, évolutive et tolérante aux pannes, d'après son expérience de travail sur les systèmes de traitement de données distribuées chez Backtype et Twitter.

La LA vise à satisfaire les besoins d'un système robuste tolérant aux pannes, à la fois contre les défaillances matérielles et les erreurs humaines, pouvant servir une large gamme de charges de travail et d'utilisation, et dans lequel des lectures et des mises à jour à faible latence sont requises. Le système résultant devrait être linéairement évolutif, et il devrait s'étendre horizontalement plutôt que verticalement. [24]

3.2 Implémentation du moteur de recommandation

3.2.1 Recommandation statique

La première partie du stage a consisté à mettre en place une recommandation statique basée sur les données disponibles dans la base de données de l'application. Deux tâches (job

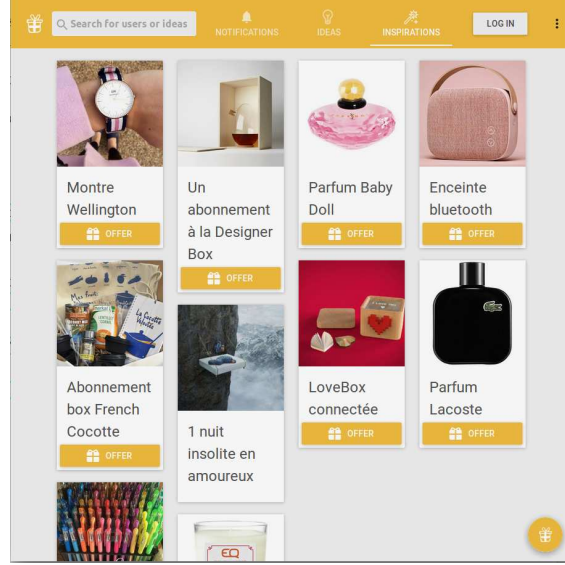


FIGURE 3.6: Interface web de l'application Pleazup

en anglais) ont été implémentés, la première assez consommatrice en terme de ressources tourne une fois tous les jours et une partie assez rapide à calculer qui utilise les valeurs pré-calculées par l'autre tâche. Un gestionnaire de tâche périodique appelle ces tâches à des intervalles réguliers. Les valeurs ainsi calculées sont utilisées pour proposer les idées de cadeaux aux utilisateurs de l'application en production actuellement. La figure 3.6 montre l'interface web de l'application avec l'onglet "Inspirations" sélectionné qui montre les idées de cadeau ordonnées à partir de l'index calculé par la procédure expliquée ci-dessus.

3.2.2 Choix du fournisseur de service Cloud

Une étude de comparaison de prix des différents fournisseurs d'hébergement Cloud a été menée avant la configuration de l'environnement serveur. Le fournisseur Digital Ocean a ainsi été retenu après cette étude. [25] La figure 3.7 montre un graphique de comparaison de prix entre les différents fournisseurs d'hébergement Cloud relativement à la quantité de mémoire vive fournie. Le choix s'est porté sur Digital Ocean du fait du prix compétitif avec une plus grande facilité de déploiement et d'utilisation en générale.

3.2.3 Filtrage collaboratif

Anatomie d'une recommandation : Algorithme de Cooccurrence 1.

$$r = [P^t P] h_p \quad (3.1)$$

r = recommandations

h_p = historique d'un utilisateur pour une action principale (achat par exemple)

P = historique de tout les actions des utilisateurs, les lignes sont les utilisateurs, les colonnes sont les articles

$[P^t P]$ = comparaison colonne à colonne utilisant la corrélation basée sur le test log-likelihood

Pratiquement tous les systèmes de recommandation de type de filtrage collaboratif existants utilisent un seul indicateur de préférence produisant une recommandation comme illustrée par la formule 3.1 ci-dessus.

Mais la théorie ne s'arrête pas là, on peut trouver une corrélation entre différents événements (cf. 3.1.6).

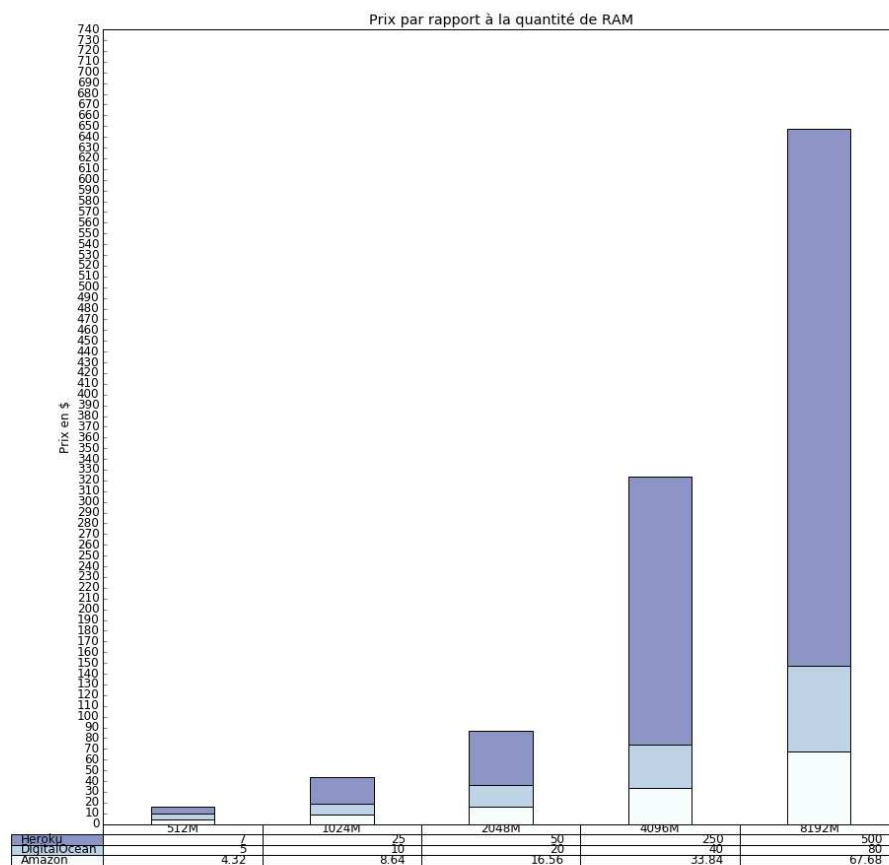


FIGURE 3.7: Comparaison de prix entre différents fournisseurs d’hébergement Cloud relativement à la quantité de mémoire vive fournie

$$r = [P^t P]h_p + [P^t V]h_v + [P^t C]h_c + \dots \quad (3.2)$$

Pratiquement tout ce que nous savons sur l'utilisateur peut être utilisé pour améliorer les recommandations -achat, vue, préférence de catégorie, préférence de localisation, préférence de périphérique, genre, ...

Déroulement du code

`SimilarityAnalysis.cooccurrences()` (cf. listing 3.4) [26] fait tout le travail pour $[P^t P]$ et $[P^t ?]$. Ca renvoie `SparkIndexedDataset` pour chaque pair de matrices $P^t P$, $P^t V$, $P^t C$, ... Ce qu'il fait consiste notamment à :

- prendre comme entrée d'une liste de matrice d'identifiants, une par type d'indicateur
- `sampleDownAndBinarize`, échantillonne aléatoirement et convertit des valeurs à $[1,0]$, en fait, il crée une version sparse de celles-ci, donc les omissions indiquent 0, seulement 1 est effectivement stocké
- `val drmAtB = drmA.t \%*\% drmB` configure les $[P^t ?]$
- `computeSimilarities` calcule llr pour chaque élément non nul de $[P^t ?]$

Listing 3.1: `org.apache.mahout.math.cf.SimilarityAnalysis.cooccurrencesIDSs`

```
/**
 * Calculates item (column-wise) similarity using the log-
 * likelihood ratio on A'A, A'B, A'C, ... and returns
 * a list of similarity and cross-similarity matrices.
 * Somewhat easier to use method, which handles the ID
 * dictionaries correctly
 *
 * @param indexedDatasets first in array is primary/A matrix
 * all others are treated as secondary
 * @param randomSeed use default to make repeatable, otherwise
 * pass in system time or some randomizing seed
 * @param maxInterestingItemsPerThing max similarities per
 * items
 * @param maxNumInteractions max number of input items per
 * item
 * @param parOpts partitioning params for drm.par(...)
 * @return a list of [[org.apache.mahout.math.indexeddataset.
 * IndexedDataset]] containing downsampled
 * IndexedDatasets for cooccurrence and cross-
 * cooccurrence
 */
def cooccurrencesIDSs(
  indexedDatasets: Array[IndexedDataset],
  randomSeed: Int = 0xdeadbeef,
  maxInterestingItemsPerThing: Int = 50,
  maxNumInteractions: Int = 500,
  parOpts: ParOpts = defaultParOpts):
  List[IndexedDataset] = {
    val drms = indexedDatasets.map(_.matrix.asInstanceOf[DrmLike
      [Int]])
    val primaryDrm = drms(0)
```

```

val secondaryDrms = drms.drop(1)
val coocMatrices = cooccurrences(primaryDrm, randomSeed,
    maxInterestingItemsPerThing,
    maxNumInteractions, secondaryDrms, parOpts)
val retIDSs = coocMatrices.iterator.zipWithIndex.map {
    case ( drm, i ) =>
        indexedDatasets(0).create(drm, indexedDatasets(0).
            columnIDs, indexedDatasets(i).columnIDs)
    }
    retIDSs.toList
}

```

Listing 3.2: org.apache.mahout.math.cf.SimilarityAnalysis.cooccurrences

```

/**
 * Calculates item (column-wise) similarity using the log-
 * likelihood ratio on A'A, A'B, A'C, ...
 * and returns a list of similarity and cross-similarity
 * matrices
 *
 * @param drmARaw Primary interaction matrix
 * @param randomSeed when kept to a constant will make
 *     repeatable downsampling
 * @param maxInterestingItemsPerThing number of similar items
 *     to return per item, default: 50
 * @param maxNumInteractions max number of interactions after
 *     downsampling, default: 500
 * @param parOpts partitioning params for drm.par(...)
 * @return a list of [[org.apache.mahout.math.drm.DrmLike]]
 *     containing downsampled DRMs for cooccurrence and
 *     cross-cooccurrence
 */
def cooccurrences(
    drmARaw: DrmLike[Int],
    randomSeed: Int = 0xdeadbeef,
    maxInterestingItemsPerThing: Int = 50,
    maxNumInteractions: Int = 500,
    drmBs: Array[DrmLike[Int]] = Array(),
    parOpts: ParOpts = defaultParOpts)
    : List[DrmLike[Int]] = {

    implicit val distributedContext = drmARaw.context

    // backend partitioning defaults to 'auto', which is often
    // better decided by calling funciton
    // todo: this should ideally be different per drm
    drmARaw.par( min = parOpts.minPar, exact = parOpts.exactPar,
        auto = parOpts.autoPar)

    // Apply selective downsampling, pin resulting matrix
    val drmA = sampleDownAndBinarize(drmARaw, randomSeed,
        maxNumInteractions)

```

```

// num users, which equals the maximum number of
// interactions per item
val numUsers = drmA.nrow.toInt

// Compute & broadcast the number of interactions per thing
// in A
val bcastInteractionsPerItemA = drmBroadcast(drmA.
  numNonZeroElementsPerColumn)

// Compute cooccurrence matrix  $A'A$ 
val drmAtAA = drmA.t \%*\% drmA

// Compute loglikelihood scores and sparsify the resulting
// matrix to get the similarity matrix
val drmSimilarityAtAA = computeSimilarities(drmAtAA, numUsers,
  maxInterestingItemsPerThing,
  bcastInteractionsPerItemA, bcastInteractionsPerItemA,
  crossCooccurrence = false)

var similarityMatrices = List(drmSimilarityAtAA)

// Now look at cross cooccurrences
for (drmBRow <- drmBs) {
  // backend partitioning defaults to 'auto', which is often
  // better decided by calling funciton
  // todo: this should ideally be different per drm
  drmARaw.par( min = parOpts.minPar, exact = parOpts.
    exactPar, auto = parOpts.autoPar)

  // Down-sample and pin other interaction matrix
  val drmB = sampleDownAndBinarize(drmBRow, randomSeed,
    maxNumInteractions).checkpoint()

  // Compute & broadcast the number of interactions per
  // thing in B
  val bcastInteractionsPerThingB = drmBroadcast(drmB.
    numNonZeroElementsPerColumn)

  // Compute cross-cooccurrence matrix  $A'B$ 
  val drmAtB = drmA.t \%*\% drmB

  val drmSimilarityAtB = computeSimilarities(drmAtB,
    numUsers, maxInterestingItemsPerThing,
    bcastInteractionsPerItemA, bcastInteractionsPerThingB)

  similarityMatrices = similarityMatrices :+
    drmSimilarityAtB

  drmB.uncache()
}

```

```

// Unpin downsampled interaction matrix
drmA.uncache()

// Return list of similarity matrices
similarityMatrices
}

```

La table 3.1 donne un exemple de données d'entrée à Universal Recommender et la table 3.2 donne une représentation sous forme de tableau de celles-ci.

TABLE 3.1: Input Events

Andrew	buy	iPhone 5
Sebastian	buy	Galaxy
Ted	buy	iPad
Sarah	buy	iPhone 5
Alexey	buy	Galaxy
Isabelle	buy	Galaxy
Andrew	view	iPhone 5
Andrew	view	Galaxy
Andrew	view	Nexus
Sebastian	view	iPhone 5
Sebastian	view	Galaxy
Sebastian	view	iPad
Sebastian	view	Surface
Ted	view	iPhone 6
Ted	view	iPad
Sarah	view	iPhone 5
Sarah	view	Galaxy
Sarah	view	Surface
Alexey	view	iPhone 6
Alexey	view	Galaxy
Alexey	view	Surface
Isabelle	view	Galaxy
Isabelle	view	Surface
Pat	view	iPhone 5
Pat	view	Galaxy
Pat	view	iPad
Pat	view	Surface

TABLE 3.2: IndexedDataset de Mahout

(a) Buy Indicator						
	iPhone 5	iPhone 6	Galaxy	Nexus	iPad	Surface
Andrew	1					
Sebastian			1			
Ted					1	
Sarah	1					
Alexey			1			
Isabelle			1			
Pat						

(b) View Indicator						
	iPhone 5	iPhone 6	Galaxy	Nexus	iPad	Surface
Andrew	1		1	1		
Sebastian	1		1		1	1
Ted		1			1	
Sarah	1		1			1
Alexey		1	1			
Isabelle			1			1
Pat	1		1		1	1

Les sorties de la procedure expliquée ci-dessous est représentée par les tableaux 3.3 et 3.4. On remarque qu'il y a trop peu d'achats à utiliser dans $llr(P^P)$ donc pas de recommandations sans données de page de visualisation !

La Log-Likelihood Ratio (LLR)

Comme mentionné ci-dessus (cf. 3.1.7) Pat Ferrel a eu l'idée d'utiliser la LLR dans Universal Recommender après des discussions avec Ted Dunning. Ce dernier a publié un article intitulé "Accurate Methods for the Statistics of Surprise and Coincidence" [27] en 1993 qui depuis a reçu de nombreuses citations avec notamment des applications dans

TABLE 3.3: Sortie de SimilarityAnalysis.cooccurrences pour P^tV

$llr(P^tV)$	IndexedDataset	Articles les plus corrélés
	Galaxy	Galaxy :1.24, iPad : 0.19, iPhone 5 : 0.19, Surface : 0.19, iPhone 6 :0.19
	iPad	iPad : 1.92, iPhone 6 : 1.24
	iPhone 5	iPhone 5 : 4.55, Nexus : 2.96, Galaxy : 0.73, Surface : 0.05

TABLE 3.4: Sortie de SimilarityAnalysis.cooccurrences pour P^tP

$llr(P^tP)$	IndexedDataset	Articles les plus corrélés
	tout	Pas de corrélations LLR significatives

le domaine de l'identification de la langue, d'espèces, ou d'auteur, la classification de documents, l'évaluation des risques d'assurance, la recommandation musicale et de vidéo.

La méthode au cœur de tous ces algorithmes est d'utiliser un score pour analyser les comptes d'événements, en particulier les comptes des événements se produisant ensemble. Les comptes que vous avez habituellement dans ces situations sont le nombre de fois où deux événements se sont produits ensemble, le nombre de fois qu'ils se sont produits avec ou sans l'autre et le nombre de fois où rien ne s'est produit.

Pour calculer le score, il est pratique de réaffirmer ces chiffres légèrement comme le nombre de fois où les événements se sont déroulés ensemble (appelons cela k_{11}), le nombre de fois que chacun s'est produit sans l'autre (appelons ces k_{12} et k_{21}) et le nombre de fois où on a observé quelque chose qui n'était aucun de ces événements (appelons cela k_{22}). Dans cette notation, la ligne ou la colonne 1 sont utilisées pour l'événement d'intérêt et la ligne ou la colonne 2 pour tout le reste.

Voici le tableau dont nous avons besoin

TABLE 3.5: Matrice de contingence

	Event A	Everything but A
Event B	A and B together (k_{11})	B, but not A (k_{12})
Everything but B	A without B (k_{21})	Neither A nor B (k_{22})

Une fois ces nombres obtenus, la partie difficile est terminée. Le calcul du score du rapport log-vraisemblance (également appelé G^2) est très simple,

$$LLR = 2 \sum(k) (H(k) - H(rowSums(k)) - H(colSums(k))) \quad (3.3)$$

Où H est l'entropie de Shannon, calculée comme la somme de $(k_{ij}/\sum(k)) \log(k_{ij}/\sum(k))$.

En R, cette fonction s'écrit comme suit :

Listing 3.3: LLR en R

```
llr <- function(k) {
  r = 2* sum(k) * (llr.H(k) - llr.H(rowSums(k)) - llr.H(
    colSums(k)))
  if (r < 0 && r > -1e-12) {
    r = 0
  }
  r
}
```

```
llr.H <- function(k) {
  N = sum(k)
  sum(k/N * log(k/N + (k==0)))
}
```

Correlated Cross-Occurrence : K-Nearest Neighbors

Dans la formule 3.2, le produit scalaire de deux vecteurs donne le cosinus de l'angle entre eux.

Le cosinus de l'angle entre deux vecteurs est largement utilisé en Machine Learning pour trouver la similarité. On le trouve alors implémenté dans presque la plupart des moteurs de recherches. [28]

$[P^t P]h_p$ et $[P^t V]h_v$ sont le résultat du produit scalaire de chaque article avec h_p (historique de l'utilisateur pour l'indicateur "purchase") et h_v (historique de l'utilisateur pour l'indicateur "view") ajoutés.

C'est exactement ce que Lucene fait :

- $[P^t P]$ est un "field" Lucene et $[P^t V]$ est un autre
- H_p est la "requête" sur le champ $[P^t P]$, nous le nommerons d'après P, comme étant le champ "purchase" (achat) et h_v est la requête sur $[P^t V]$ ou le champ "view" (vue).
- Lucene fait la somme et trie les résultats produisant r , une liste classée de "documents" ou dans notre cas d'articles recommandés
- Cela donne le KNN à l'historique de l'utilisateur c'est-à-dire des articles qu'ils ont une affinité pour

3.2.4 Règles métier

Certaines règles métier ont été dictées par le business model et les spécificités de l'application.

D'abord, en ce qui concerne le business model, les idées de cadeau affiliées devaient être mises en avant après avoir calculer la recommandation.

Ceci a été implémenté dans le fichier de configuration d' Universal recommender à partir de biais appliqués sur des articles ayant des propriétés particuliers.

Le deuxième règle métier que l'on devait tenir en compte était de ne montrer dans les inspirations que les idées visibles et qui avaient des images associées.

Le premier a été implémenté en amont en envoyant vers l'event server que les idées qui sont visibles.

Le deuxième parcontre a été mis en œuvre une fois encore avec la configuration du biais dans le fichier de configuration engine.json d'Universal Recommender. [29]

Biais

Les biais peuvent être considérés en tant que multiplicateur pour le score des éléments qui répondent à la condition, donc si $\text{bias} = 2$, et l'article 1 répond à la condition, on multiplie le score de l'élément 1 par la valeur donnée au biais.

Après l'application de tous les biais, les recommandations sont renvoyées classées par score.

L'effet du biais est de :

Bias > 0 à <1 : diminuer le classement pour les éléments qui correspondent à la condition

Bias = 1 : sans effet

Bias > 1 : augmenter le classement pour les éléments qui correspondent à la condition.

Bias = 0 : exclure tous les éléments qui correspondent à la condition

Bias < 0 : Un biais négatif ne renverra que les éléments qui répondent à la condition, donc en d'autres termes, il filtre ceux qui ne répondent pas aux conditions.

Engine.json

Ce fichier permet à l'utilisateur de décrire et de définir des paramètres qui contrôlent les opérations du moteur ("engine"). De nombreuses valeurs ont des valeurs par défaut, de sorte que ce qui suit peut être considéré comme le minimum pour une application e-commerce avec un seul événement "buy".

Listing 3.4: Valeurs par défaut simples

```
{
  "comment": "This config file uses default settings for all
    but the required values see README.md for docs",
  "id": "default",
  "description": "Default settings",
  "engineFactory": "org.template.RecommendationEngine",
  "datasource": {
    "params": {
      "name": "datasource-name",
      "appName": "handmade",
      "eventNames": ["purchase", "view"]
    }
  },
  "sparkConf": {
    "spark.serializer": "org.apache.spark.serializer.
      KryoSerializer",
    "spark.kryo.registrator": "org.apache.mahout.sparkbindings
      .io.MahoutKryoRegistrator",
    "spark.kryo.referenceTracking": "false",
    "spark.kryoserializer.buffer.mb": "300",
    "spark.kryoserializer.buffer": "300m",
    "spark.executor.memory": "4g",
    "es.index.auto.create": "true"
  },
  "algorithms": [
    {
      "comment": "simplest setup where all values are default,
        popularity based backfill, must add eventsNames",
      "name": "ur",
      "params": {
        "appName": "handmade",
        "indexName": "urindex",
        "typeName": "items",
        "comment": "must have data for the first event or the
          model will not build, other events are optional",
        "eventNames": ["purchase", "view"]
      }
    }
  ]
}
```

3.2.5 Filtrage basé sur le contenu

Out of the box, Prediction IO utilise les propriétés des articles pour faire du filtrage basé sur le contenu. Ces propriétés sont utilisées dans une requête pour booster certains éléments recommandés. Le boost peut être très élevé ou négatif pour filtrer puis faire un filtrage collaboratif ce qui a pour effet de filtrer les articles qui correspondent exactement aux propriétés de l'élément contextuel.

Il existe d'autres façons d'utiliser le contenu des articles et faire du filtrage collaboratif basé sur le contenu, comme en créant des topics LDA [30] et en attribuant des identifiants comme s'ils étaient des tags créés par des humains. Au fur et à mesure que les topics des contenus changent, les IDs changent et sont toujours un indicateur de l'historique des sujets préférés par l'utilisateur. Ajouter Word2Vec [31] à LDA devrait permettre de créer des topics de très haute qualité.

D'après Pat Ferrel, le combo de w2v + LDA peut être combiné avec l'UR existant, mais serait un modèle complémentaire distinct pour créer des événements enrichissants pour l'UR. [32]

3.2.6 Implémentation et mise en production

Un job² a été implémenté dans le back-end Parse Server de *Pleazup* pour envoyer en batch ainsi qu'en temps réel les actions des utilisateurs ainsi que les mis à jour qui peuvent se faire sur les articles de cadeau. Pour ce faire, on a utilisé `predictionio-driver` [34] qui est un pilote prenant en charge à la fois la syntaxe `callback` et la syntaxe basée sur les `promises` pour accéder de manière asynchrone aux APIs exposés par le serveur d'événements et le moteur de recommandation de PredictionIO.

L'architecture du système qui a été mis en place est représentée par la figure 3.8.

La machine du milieu comme représentée dans la figure 3.8 est hébergée chez DigitalOcean et on y a installé PredictionIO. Tout le processus a été documenté par étape sur le wiki du repository du projet sur Atlassian Bitbucket. [35]

Cette machine héberge le serveur d'événement et répond en temps réel aux requête provenant des clients qui sont relayées par le back-end Parse de *Pleazup*.

La troisième machine à l'extrême droite sur l'illustration de la figure 3.8 est le master d'un cluster Spark. On l'utilise pour entraîner le modèle prédictif qui sera déployé sur la machine du milieu. La machine contient aussi une installation de la chaîne d'outils de PredictionIO qui est utilisée lors la phase d'entraînement du modèle prédictif.

3.3 Evaluation et test de montée en charge

3.3.1 Evaluation

Pour évaluer les recommandations il faudra diviser les données en données d'entraînement et en données de test. Pour mieux imiter le monde réel du mieux possible dans les tests hors ligne, il faudra prendre une partie des données les plus récentes dans l'historique des interactions utilisateur comme données de test.

Dans son article intitulé *Toward a New Protocol to Evaluate Recommender Systems* [1], Frank Meyer propose un protocole et présente quelques mesures qui peuvent être utilisées pour évaluer la performance d'un système de recommandation selon la fonction attendue du système.

Un moteur PredictionIO est instancié avec un ensemble de paramètres, ces paramètres déterminent quel algorithme est utilisé ainsi que les paramètres de l'algorithme. Cela

2. Un job est une tâche de fond que l'on peut exécuter et suivre le status dans le tableau de bord de Parse Server [33]

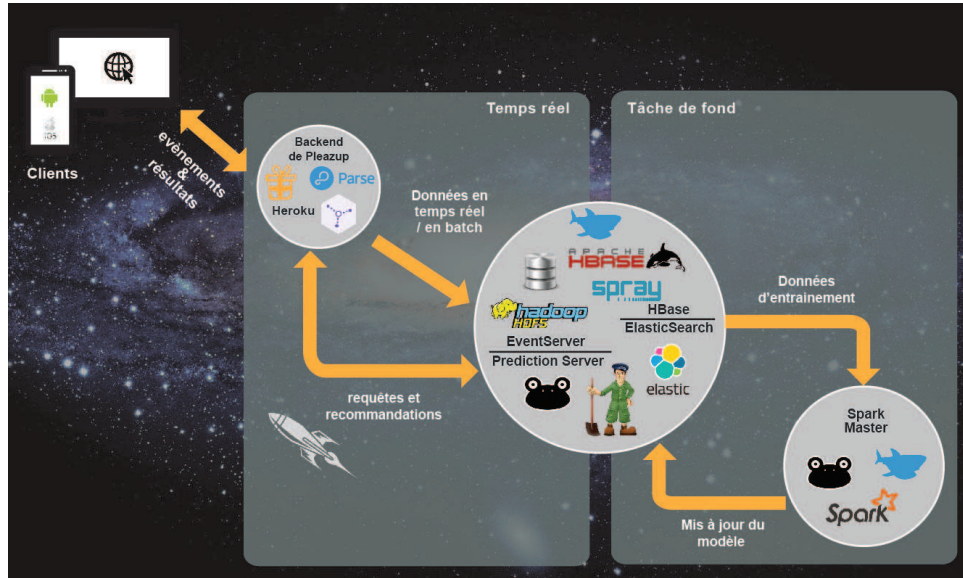


FIGURE 3.8: Architecture système de l'écosystème technologique avec le système de recommandation

soulève naturellement la question de savoir comment choisir le meilleur ensemble de paramètres.

Le module d'évaluation de PredictionIO permet de rationaliser le processus de test de nombreux points dans les paramètres du moteur et de déployer le meilleur en utilisant des méthodes de validation croisée statistiquement correctes.

Il y a deux éléments clés :

Moteur

C'est l'objet de l'évaluation. Au cours de l'évaluation, en plus du mode train et deploy que nous décrivons dans les sections précédentes, le moteur génère également une liste de points de données de test. Ces points de données sont une séquence de tuples de requête et de résultats réels. Les requêtes sont envoyées au moteur et le moteur répond avec un résultat prédit, de la même manière que le moteur répond à une requête.

Évaluateur

L'évaluateur rejoint la séquence de la requête, du résultat prédit et du résultat réel et évalue la qualité du moteur. PredictionIO permet de mettre en œuvre n'importe quelle métrique avec seulement quelques lignes de code.

3.3.2 Test de montée en charge

Pour la phase de test de montée en charge, on prévoit d'utiliser les données existantes et d'introduire des doublons pour ensuite tester les différentes parties du système de recommandation quant à leur résistance à la montée en charge.

Il s'agira dans un premier temps de tester le serveur d'événements en lui envoyant une grande masse d'événements provenant d'utilisateurs fictifs.

Pour cela, PredictionIO offre la notion de **channel** [36] qui peut être utilisée pour séparer ces données factices des données réelles générées par les utilisateurs de l'application.

Conclusion

Au terme de ces 5 mois et quelques de stage, j'ai pu approfondir et ancrer mon savoir-faire sur les systèmes de recommandation et dans le domaine du Machine Learning en général.

La première partie définie par le sujet s'est portée sur du développement Web. Domaine dans lequel j'ai déjà eu une expérience professionnelle préalable. Ce qui m'a permis de finir et de mettre en production la recommandation statique assez vite.

Ensuite, on s'est attelé à la comparaison des différentes offres Cloud pour héberger le futur moteur de recommandation. En même temps, on s'est aperçu que la plate-forme PredictionIO semblait bien répondre aux besoins que l'on recherchait et la communauté est active dans le développement ainsi que sur les listes de diffusion.

Après que le choix du fournisseur Cloud s'est fait, qui s'est porté sur Digital Ocean, j'ai commencé à installer sur le serveur un environnement PredictionIO après l'avoir essayé sur ma machine et avoir réussi à faire passer le test d'intégration.

À la suite de cela, on a enchaîné avec l'intégration avec le backend Parse de *Pleazup*. Une étape qui a eu ses péripéties à cause du caractère distribué du système. Après que cela ait été fait, nous avons configuré le moteur pour tenir compte des règles métier du réseau social *Pleazup*.

Ensuite, en s'apercevant que la phase d'entraînement du moteur consommait beaucoup de ressources mémoire qui n'étaient requises que pendant cette phase, on a décidé de faire l'entraînement sur une autre machine avec beaucoup de mémoire vive. On a alors pu mettre en place à l'heure actuelle un système de recommandation opérationnel tenant compte des événements temps réel et répondant aux requêtes des clients relayées par le back-end de *Pleazup*. L'entraînement a également été automatisé pour permettre de déployer un nouveau modèle en production sans interruption de service.

Pendant le temps qui nous reste, on va essayer de trouver une solution pour intégrer les données texte et images associés aux articles de cadeau pour améliorer et renforcer le système de recommandation qui est en place. A la suite de cela, se fera la mise en place d'un protocole d'évaluation et de test de montée en charge pour suivre et faire évoluer le système.

Pour conclure, ce stage a répondu à mes attentes et m'a mis face à des défis qui feront de moi un meilleur ingénieur. J'ai aussi pris goût et ai été inspiré par le côté algorithmique du Machine Learning au cours du stage. Domaine dans lequel j'envisage de continuer ma carrière.

Bibliographie

- [1] Frank Meyer, Françoise Fessant, Fabrice Clérot, and Éric Gaussier. Toward a new protocol to evaluate recommender systems. In *Proceedings of the Workshop on Recommendation Utility Evaluation : Beyond RMSE, RUE 2012, Dublin, Ireland, September 9, 2012*, pages 9–14, 2012.
- [2] Frank Meyer. *Systèmes de recommandation dans des contextes industriels. (Recommender systems in industrial contexts)*. PhD thesis, Grenoble Alpes University, France, 2012.
- [3] Marcel Mauss. Essai sur le don forme et raison de l’échange dans les sociétés archaïques. *L’Année sociologique (1896/1897-1924/1925)*, 1 :30–186, 1923.
- [4] Ce que nos cadeaux disent de nous. <http://noel.psychologies.com/L-art-du-bon-cadeau/Ce-que-nos-cadeaux-disent-de-nous>. Accessed : 2017-06-04.
- [5] Technopole de la réunion. https://fr.wikipedia.org/wiki/Technopole_de_La_R%C3%A9union. Accessed : 2017-03-10.
- [6] Site officiel de la technopole de la réunion. <http://technopole-reunion.com/>. Accessed : 2017-03-11.
- [7] Laboratoire d’informatique et de mathématiques (lim) de l’université de la réunion. <http://lim.univ-reunion.fr/organisation/presentation/>. Accessed : 2017-05-03.
- [8] Stage r&d master 2 en startup développement d’un moteur de recommandation. <http://lim.univ-reunion.fr/staff/fred/M2info/16-17/prop-stages/Pleazup.pdf>. Accessed : 2017-03-11.
- [9] Xiaoyuan Su and Taghi M. Khoshgoftaar. A survey of collaborative filtering techniques. *Adv. in Artif. Intell.*, 2009 :4 :2–4 :2, January 2009.
- [10] K Nageswara Rao. Application domain and functional classification of recommender systems—a survey. *DESIDOC Journal of Library & Information Technology*, 28(3) :17, 2008.
- [11] Sebastian Schelter, Christoph Boden, and Volker Markl. Scalable similarity-based neighborhood methods with mapreduce. In *Proceedings of the sixth ACM conference on Recommender systems*, pages 163–170. ACM, 2012.
- [12] Roberto J Bayardo, Yiming Ma, and Ramakrishnan Srikant. Scaling up all pairs similarity search. In *Proceedings of the 16th international conference on World Wide Web*, pages 131–140. ACM, 2007.
- [13] Badrul Sarwar, George Karypis, Joseph Konstan, and John Riedl. Item-based collaborative filtering recommendation algorithms. In *Proceedings of the 10th international conference on World Wide Web*, pages 285–295. ACM, 2001.
- [14] More b2d : Tappingstone wants to tap the need for data scientists with its user behavior predictions-as-a-service. <https://techcrunch.com/2012/10/18/>

- more-b2d-tappingstone-wants-to-tap-the-need-for-data-scientists-with-its-user-beha
 Accessed : 2017-05-31.
- [15] Salesforce signs a definitive agreement to acquire predictionio. <http://blog.prediction.io/salesforce-signs-a-definitive-agreement-to-acquire-predictionio/>. Accessed : 2017-05-31.
 - [16] Salesforce's predictionio donated to the apache software foundation. <https://www.salesforceiq.com/blog/salesforces-predictionio-donated-to-the-apache-software-foundation>. Accessed : 2017-05-31.
 - [17] Fay Chang, Jeffrey Dean, Sanjay Ghemawat, Wilson C. Hsieh, Deborah A. Wallach, Mike Burrows, Tushar Chandra, Andrew Fikes, and Robert E. Gruber. Bigtable : A distributed storage system for structured data. *ACM Trans. Comput. Syst.*, 26(2) :4 :1–4 :26, June 2008.
 - [18] Predictionio - a quick intro. <http://predictionio.incubator.apache.org/start/>. Accessed : 2017-06-01.
 - [19] Predictionio - learning dase. <http://predictionio.incubator.apache.org/customize/>. Accessed : 2017-06-01.
 - [20] <http://actionml.com/docs/ur>. Accessed : 2017-03-12.
 - [21] Correlated cross-occurrence (cco) : How to make data behave as we want. <http://www.actionml.com/blog/cco>. Accessed : 2017-05-03.
 - [22] The universsal recommender with cco. <https://docs.google.com/presentation/d/1MzIGFsATNeAYnLfoR6797ofcLeFRKSX7KB8GAYNtNPY/>. Accessed : 2017-05-27.
 - [23] Llr and other similarity metrics graph in the universal recommender with cco slide. <https://groups.google.com/forum/#!topic/actionml-user/wVP-R3sQgvk>. Accessed : 2017-05-04.
 - [24] Lambda architecture. <http://lambda-architecture.net/>. Accessed : 2017-05-02.
 - [25] Digitalocean. <https://www.digitalocean.com/>. Accessed : 2017-03-12.
 - [26] Mahout math cf similarityanalysis. Accessed : 2017-05-29.
 - [27] Ted Dunning. Accurate methods for the statistics of surprise and coincidence. *COMPUTATIONAL LINGUISTICS*, 19(1) :61–74, 1993.
 - [28] Apache lucene search similarity. https://lucene.apache.org/core/3_0_3/api/core/org/apache/lucene/search/Similarity.html. Accessed : 2017-05-30.
 - [29] Universal recommender tuning. http://actionml.com/docs/ur_config. Accessed : 2017-05-30.
 - [30] David M. Blei, Andrew Y. Ng, and Michael I. Jordan. Latent dirichlet allocation. *Journal of Machine Learning Research*, 3 :993–1022, 2003.
 - [31] Tomas Mikolov, Ilya Sutskever, Kai Chen, Gregory S. Corrado, and Jeffrey Dean. Distributed representations of words and phrases and their compositionality. In *Advances in Neural Information Processing Systems 26 : 27th Annual Conference on Neural Information Processing Systems 2013. Proceedings of a meeting held December 5-8, 2013, Lake Tahoe, Nevada, United States.*, pages 3111–3119, 2013.
 - [32] Ur is hybrid collaborative filtering and content-based recommender. Accessed : 2017-06-05.
 - [33] Cloud jobs. <http://docs.parseplatform.org/cloudcode/guide/#cloud-jobs>. Accessed : 2017-06-04.
 - [34] predictionio-driver. <https://github.com/asafyish/predictionio-driver>. Accessed : 2017-06-04.

- [35] Atlassian bitbucket. <https://bitbucket.org/product>. Accessed : 2017-06-04.
- [36] Predictionio - collecting and analyzing data - channel. <https://predictionio.incubator.apache.org/datacollection/channel/>. Accessed : 2017-06-06.
- [37] Apache predictionio (incubating). <http://predictionio.incubator.apache.org/index.html>. Accessed : 2017-03-13.

Annexes

Annexe A : Accomplissements personnels de l'étudiant

MOOC : Machine Learning, Coursera

J'ai suivi le MOOC *Machine Learning* enseigné par **Pr. Andrew Ng**, Associate Professor à *Stanford University* avant de début du stage, depuis que j'ai commencé à m'intéresser au domaine du *Machine Learning*.

Cette classe de *Machine Learning* était la classe qui avait commencé Coursera.

Dans cette classe, les étudiants prennent connaissance des meilleures techniques de *Machine Learning* et acquièrent de la pratique dans leur mise en œuvre. Plus important, ils ne vont pas seulement prendre connaissance des fondements théoriques, mais également acquérir le savoir-faire requis pour rapidement et efficacement appliquer ces techniques à de nouveaux problèmes. Finalement, les étudiants apprennent les meilleures pratiques en œuvre dans la Silicon Valley se rapportant au *Machine Learning* et à l'intelligence artificielle.

Cette classe contient une large introduction au *Machine Learning*, l'analyse de données et la reconnaissance de formes.

Le contenu inclut :

- (i) Apprentissage supervisé (algorithmes paramétriques / non paramétriques, machines à vecteur de support, noyaux, réseaux neuronaux).
- (ii) Apprentissage non supervisé (partitionnement de données, réduction de dimension, système de recommandation, l'apprentissage profond).
- (iii) Bonnes pratiques en *Machine Learning* (bias, variance ; processus d'innovation en *Machine Learning* et IA).

La classe décrit également plusieurs cas d'usage, ainsi l'étudiant apprend également comment appliquer les algorithmes d'apprentissage pour construire des robots intelligents (perception, contrôle), compréhension de texte (recherche web, anti-spam), vision par ordinateur, informatique médicale, signaux sonores, base de données sémantiques, et d'autres domaines.

Contribution à des projets Open Source

J'ai contribué 2 pull requests à la documentation de 2 projets que j'ai eu à utiliser au cours de mon stage.

apache/incubator-predictionio :livedoc

<https://github.com/apache/incubator-predictionio/pull/384>

asafyish/predictionio-driver README

<https://github.com/asafyish/predictionio-driver/pull/15>

Les deux ont été fusionnés au projet associé.